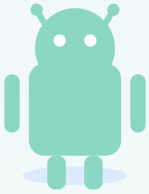
MCA SEMESTER 3 • MOBILE APPLICATION DEVELOPMENT



Basics of Android

A Simple, Easy-to-Understand Study Guide

Topics Covered:

Android SDK & Features • Development Tools • Application Basics

Android Manifest • Externalizing Resources • Application Lifecycle • Activities

📑 Table of Contents

TOPIC 1

1. Introduction to Android

Android is like a "brain" (operating system) for touch-screen devices such as mobile phones, tablets, smart TVs, and smart watches. Just like Windows works on a laptop, Android works on your mobile phone. It was made by a company called **Android Inc.**, which was later bought by **Google** in 2005.

THINK OF IT LIKE THIS

Your phone is like a body, and Android is the brain that controls everything — calling, messaging, camera, apps, everything!

Why is Android so Popular?

- **Free and Open Source** – Anyone can use and modify Android's code for free.
- **Works on many devices** – Cheap phones to expensive phones, all can run Android.
- **Millions of Apps** – Play Store has apps for everything (games, study, chatting, etc).
- **Supported by Google** – Regular updates and strong community support.

Android Architecture (How Android is Built Inside)

Android is built in layers, like a cake. Each layer has its own job. Let's understand each layer from bottom to top:

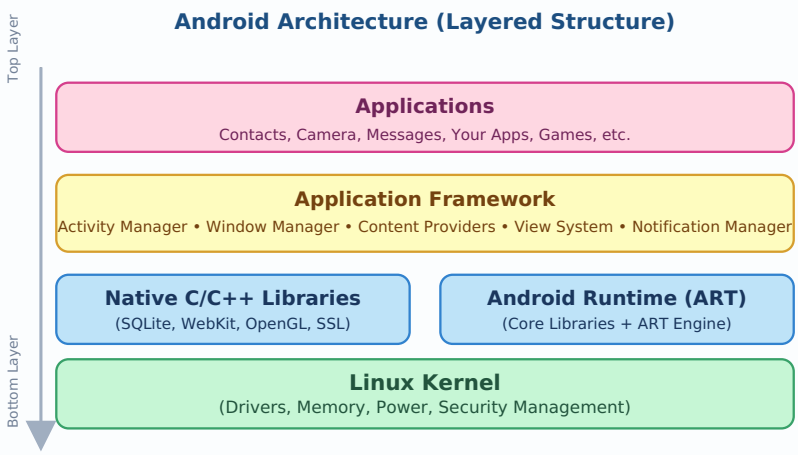


Fig 1.1 — Android Architecture Stack (from hardware level to user-facing apps)

Layer	What it Does (in simple words)
Linux Kernel	The foundation. Talks directly to the hardware (camera, Wi-Fi, battery, etc.)
Native Libraries	Ready-made C/C++ code for things like databases (SQLite) and graphics (OpenGL)
Android Runtime (ART)	Runs your app's code and converts it into something the phone understands
Application Framework	Provides tools (APIs) that developers use to build apps easily
Applications	The apps you actually see and use — WhatsApp, Camera, your own app, etc.

EXAMPLE

When you take a photo: the **Application** layer (Camera app) uses the **Framework** to open the camera, which uses **Native Libraries** to process the image, and the **Linux Kernel** actually talks to the real camera hardware.

TOPIC 2

2. Android SDK and Its Features

SDK means **Software Development Kit**. It is a box full of tools that helps a developer build, test, and run Android apps. Without the SDK, you cannot create an Android app.

SIMPLE ANALOGY

If building an app is like cooking a dish, the SDK is your **full kitchen kit** — knives, oven, ingredients holder, recipe book — everything you need to cook (build) the app.

What's Inside the Android SDK?

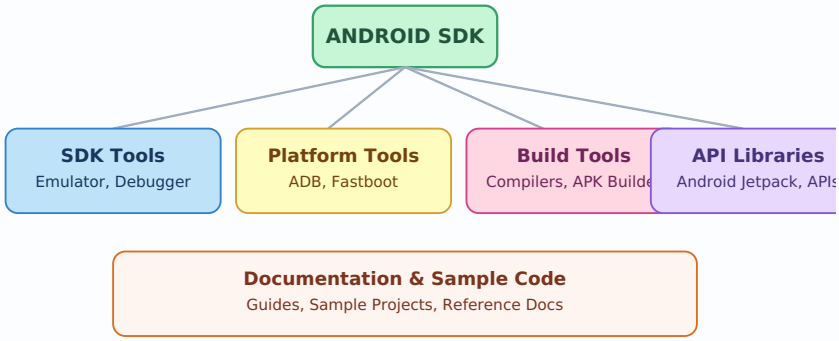


Fig 2.1 — Main components inside the Android SDK

Key Features of Android SDK

- **Emulator** – Lets you run and test your app on a virtual phone on your computer, without needing a real phone.
- **Debugger** – Helps you find and fix mistakes (bugs) in your code.
- **ADB (Android Debug Bridge)** – A tool to communicate between your computer and an Android device.
- **Ready-made Libraries** – Pre-written code for common features (camera, maps, notifications).
- **Sample Projects** – Example apps you can learn from.
- **API Levels** – Each Android version (like Android 13, 14) has its own set of features called an API level.

EXAMPLE

A student wants to build a "Calculator App". Using the SDK's **emulator**, they can run the calculator on a virtual phone screen on their laptop itself, before installing it on a real device.

TOPIC 3

3. Introduction to Development Features

Android provides many built-in features that make app development easier. These are called **development features** because they help developers build powerful apps quickly without writing everything from scratch.

Important Development Features

Feature	Simple Explanation
Activities	Each screen of your app (like Login screen, Home screen) is an Activity.
Services	Tasks that run in the background, like playing music while you use another app.
Broadcast Receivers	Listens for system-wide announcements, like "battery low" or "Wi-Fi connected".
Content Providers	Lets apps share data with each other, like accessing your Contacts list.
Intents	Messages used to move between screens or apps (like opening the Camera app from your app).
Notifications	Small alert messages shown outside the app, like a WhatsApp message popup.
Views & Layouts	Building blocks of the User Interface — buttons, text boxes, images.

WHY THIS MATTERS

These features are like **LEGO blocks**. A developer just picks the right blocks (Activity, Service, Intent, etc.) and puts them together to build a complete app — instead of building everything from zero.

Example: How features work together

Imagine a Music Player app:

- The **Activity** shows the song list screen.
- A **Service** keeps playing the song even when you switch to another app.
- A **Notification** shows the currently playing song in the status bar.
- An **Intent** is used when you tap a song to open the "Now Playing" screen.

TOPIC 4

4. Developing for Android — Mobile & Embedded Devices

Android is not only for mobile phones. It also runs on many other small devices. This is called developing for **mobile and embedded devices**.

Difference Between Mobile Development and Embedded Development

Mobile Device Development	Embedded Device Development
For phones and tablets	For smaller, special-purpose devices (smartwatch, TV, car system)
Bigger screen, touch input	Small screen or no screen, limited input (buttons, voice, gestures)
More memory and battery available	Limited memory, battery, and processing power
Example: WhatsApp, Instagram	Example: Fitness tracker app, Car dashboard app

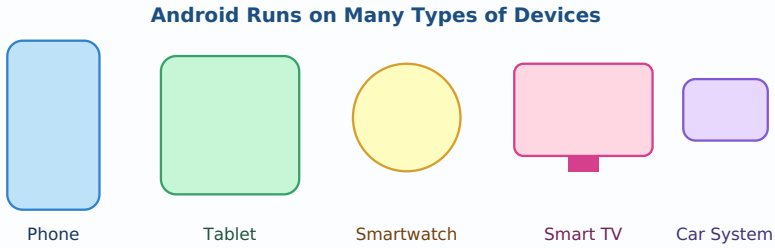


Fig 4.1 — Android is used across mobile and embedded devices

Challenges in Embedded Development

- **Less Memory/Storage** – Apps must be lightweight and efficient.
- **Different Screen Sizes** – Apps must adjust for round watch screens, big TV screens, tiny screens.
- **Different Input Methods** – Some embedded devices don't have a touchscreen; they use buttons, remote controls, or voice.
- **Battery Optimisation** – Embedded devices often run on small batteries, so apps must save power.

EXAMPLE

A fitness band app must use very little battery because the band's battery is tiny compared to a phone's battery.

TOPIC 5

5. Android Development Tools

To build Android apps, we need special software tools. These tools help us write code, design screens, test the app, and fix errors.

Main Android Development Tools

1. Android Studio

This is the **official tool (IDE – Integrated Development Environment)** used to build Android apps. It is like MS Word, but for writing app code instead of documents. It includes a code editor, design tools, and testing tools — all in one place.

2. Android Virtual Device (AVD) / Emulator

This creates a **fake (virtual) mobile phone** on your computer screen so you can test your app without needing a real phone.

3. ADB (Android Debug Bridge)

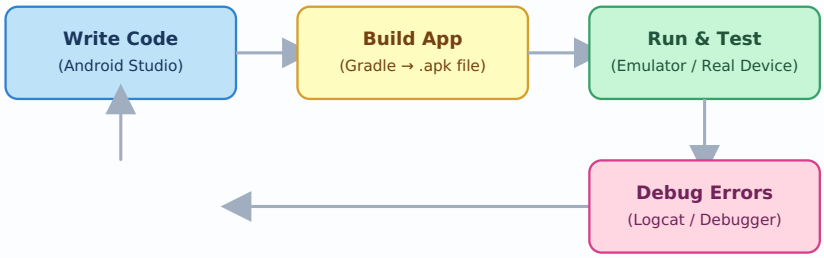
A command-line tool that lets your computer "talk" to a real or virtual Android device — for installing apps, viewing logs, and debugging.

4. Gradle

This is a **build tool**. It takes all your code, images, and resources and packages them into a final installable file called an `.apk` (or `.aab`).

5. Logcat

A tool inside Android Studio that shows real-time messages and errors while your app is running — very useful for debugging.



This cycle repeats until the app works perfectly (fix and re-test)

Fig 5.1 — The App Development Cycle using Android Tools

Tool	Purpose
Android Studio	Main editor to write and design the app
AVD / Emulator	Virtual phone to test app without a real device
ADB	Connects computer to device for installing/debugging
Gradle	Builds the project into an installable APK file
Logcat	Shows live logs and errors during testing

TOPIC 6

6. Creating Applications using Android

Let's understand the simple steps of how a real Android application is created — from an idea to a working app.

Fig 6.1 — Steps to Create an Android Application

Step-by-Step Explanation

1. **Plan the idea** – Decide what the app will do (e.g., a To-Do list app).
2. **Create a New Project** – Open Android Studio and choose a project template.
3. **Design the UI (User Interface)** – Add buttons, text, images using XML layout files.
4. **Write the Logic** – Use Java or Kotlin to make buttons and features actually work.
5. **Test the App** – Run it on an emulator/real device, fix bugs found using Logcat.
6. **Publish** – Upload the finished APK/AAB file to the Google Play Store.

EXAMPLE

To make a simple "To-Do List" app: You design a screen with a text box and an "Add" button (UI), write code so tapping "Add" saves the task (logic), test if tasks are saved correctly, then publish it.

NOTE

Android apps are mainly written in **Java** or **Kotlin** (Kotlin is Google's preferred language today), and screens are designed using **XML**.

TOPIC 7

7. Basics of an Android Application

Every Android app is made up of a few basic building blocks. Let's understand each one in very simple words.

The 4 Main Building Blocks (Components)

Component	What it Means	Real-life Example
Activity	One single screen of the app	Login screen, Home screen
Service	Work that runs in the background, without a screen	Playing music while chatting
Broadcast Receiver	Waits and reacts to system messages	Showing alert when battery is low
Content Provider	Shares data between different apps	An app accessing your phone's Gallery photos

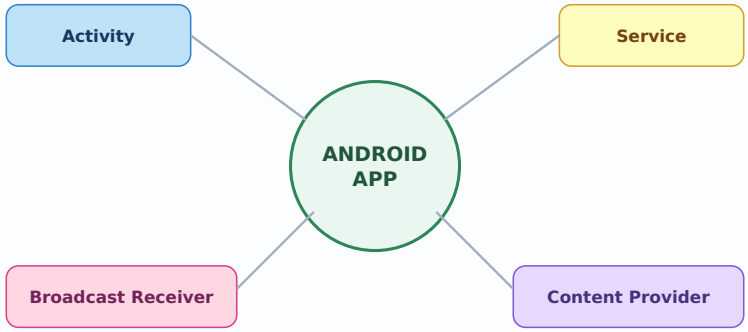


Fig 7.1 — The Four Main Components of an Android Application

Basic Project Structure

When you create a new Android project, it comes with some default folders and files:

```
MyApp/
 ├── manifests/
 │     └── AndroidManifest.xml     (App's identity card)
 ├── java/
 │     └── MainActivity.java        (Your code / logic)
 ├── res/                            (Resources - non-code content)
 │     ├── layout/                  (Screen designs - XML files)
 │     ├── values/                  (Colors, Strings, Styles)
 │     ├── drawable/                (Images, Icons)
 │     └── mipmap/                  (App Launcher Icons)
 └── build.gradle                   (Build settings)
```

IN SIMPLE WORDS

java/ folder = your app's brain (logic).
res/ folder = your app's face (design/looks).
AndroidManifest.xml = your app's ID card (tells Android what the app contains).

TOPIC 8

8. Introduction to the Manifest File

The **AndroidManifest.xml** file is like the **ID card / passport** of your Android app. Before your app can run, Android reads this file first to understand:

- What is the app's name and package (unique identity)?
- What screens (Activities) does it have?
- What permissions does it need (like Camera, Internet, Location)?
- Which screen should open first when the app starts?

SIMPLE ANALOGY

Just like your **Aadhar card** tells others your name, address, and details about you — the **Manifest file** tells the Android system everything about your app.

Example of a Simple Manifest File

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myfirstapp">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="My First App"
        android:theme="@style/AppTheme">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

Important Tags Explained

Tag	Meaning in Simple Words
`<manifest>`	The root tag; contains everything about the app
`package`	The app's unique identity (like a fingerprint), e.g., com.example.myapp
`<uses-permission>`	Lists permissions the app needs, like Camera or Internet access
`<application>`	Holds all app-level info like icon, name, and theme
`<activity>`	Declares one screen of the app
`<intent-filter>`	Tells Android which activity should open first (the Launcher/Main screen)

EXAMPLE

If a Weather App needs to fetch data from the internet, it MUST declare `<uses-permission android:name="android.permission.INTERNET" />` in the Manifest — otherwise the app will crash or not work.

REMEMBER

Every single Activity (screen) you create in your app **MUST** be declared inside the Manifest file, or Android won't know it exists!

TOPIC 9

9. Externalizing Resources

Externalizing resources means keeping things like text, images, colors, and sizes **separate from your code**, in special XML files inside the `res/` folder — instead of writing them directly (hard-coding) inside your Java/Kotlin code.

> **SIMPLE ANALOGY**
> Imagine a recipe book where ingredients are written on a separate card. If you want to change sugar quantity, you just edit the card — you don't rewrite the whole recipe. Externalized resources work the same way!

Why Externalize Resources?

- **Easy to update** – Change text/colors without touching the code.
- **Supports Multiple Languages** – Easily translate app text (English, Hindi, Gujarati, etc.)
- **Supports Multiple Screens** – Different images/sizes for different screen sizes (phone, tablet).
- **Cleaner Code** – Keeps design details out of the logic code.

Types of Resources

Resource Folder	Used For	Example File
values/strings.xml	All text used in the app	App name, button labels
values/colors.xml	Colour codes used in the app	Primary colour, background colour
values/dimens.xml	Sizes/dimensions like padding, text size	Button height, margin
drawable/	Images and icons	Logo, background image
layout/	Screen designs	activity_main.xml

Example: Without vs With Externalizing

✕ Without Externalizing (Bad Practice)

```
Button btn = new Button(this);
btn.setText("Submit");
btn.setTextColor(Color.parseColor("#FF0000"));
```

✓ With Externalizing (Good Practice)

```
<!-- strings.xml -->
<string name="submit_btn">Submit</string>

<!-- colors.xml -->
<color name="btn_color">#FF0000</color>
```

Then in your layout XML, you simply use:

```
<Button
    android:text="@string/submit_btn"
    android:textColor="@color/btn_color" />
```

> **EXAMPLE**
> If your app supports both English and Hindi, you can create `values/strings.xml` (English) and `values-hi/strings.xml` (Hindi). Android automatically picks the correct one based on the phone's language setting!

TOPIC 10

10. Application Lifecycle

The **Application Lifecycle** describes the different stages an Android app goes through — from the moment it starts, to when it goes into the background, and finally when it is closed or removed from memory.

SIMPLE ANALOGY

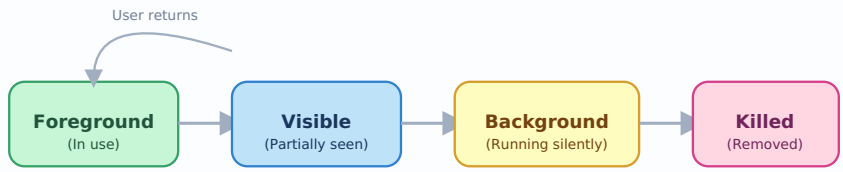
Just like a human life has stages — birth, growth, old age, death — an Android app also has stages: **Start → Run → Pause → Stop → Destroy**.

Why is the Lifecycle Important?

- Mobile devices have **limited memory**, so Android needs to manage which apps stay active and which are closed.
- Helps save the user's data/progress when they leave the app suddenly (e.g., a phone call comes in).
- Helps save battery by stopping unused background processes.

Application Process States

State	Meaning
Foreground / Active	App is currently on screen and being used by the user
Visible	App is partially visible but not in direct focus (e.g., a dialog box is on top)
Background / Service	App is not visible but still running some tasks (e.g., music playing)
Empty / Killed	App is not running; Android may remove it from memory to free up space



Android moves the app through these states as the user switches apps or the system needs memory

Fig 10.1 — Application Process States in Android

EXAMPLE

You are chatting on WhatsApp (Foreground). You press Home button to check another app — WhatsApp goes to Background but keeps running to receive new messages. If you don't use it for a long time and memory is needed, Android may Kill it completely.

TOPIC 11

11. Android Activities

An **Activity** is one single screen in an Android app with a user interface. If your app has a Login screen, Home screen, and Settings screen — each one of these is a separate **Activity**.

SIMPLE ANALOGY

Think of an app like a **book**, and each Activity is like one **page** of that book. To move from page 1 (Login) to page 2 (Home), you "turn the page" — in Android, this is done using something called an **Intent**.

The Activity Lifecycle

Just like the whole app has a lifecycle, each individual Activity (screen) also goes through its own set of stages. These stages are controlled using special methods (callbacks):

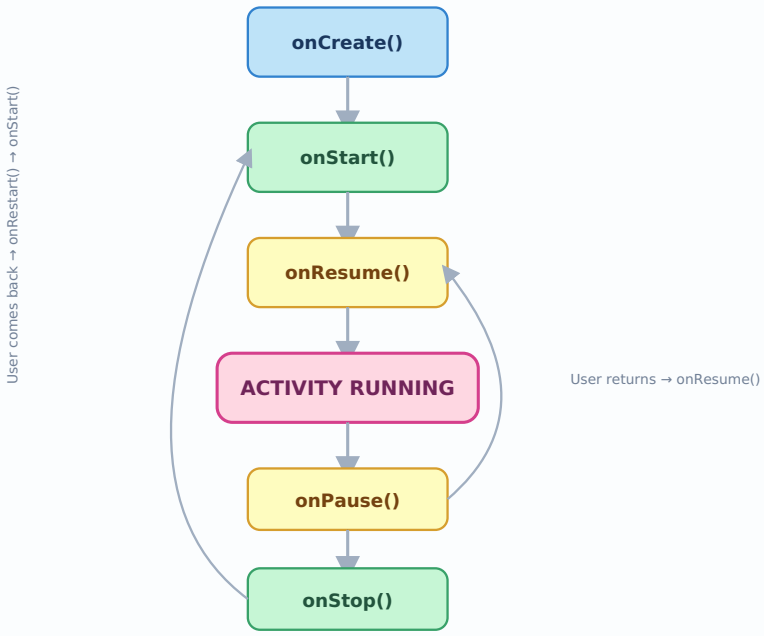


Fig 11.1 — Android Activity Lifecycle Diagram

Activity Lifecycle Methods Explained

Method	When it is Called	Simple Meaning
`onCreate()`	When the Activity is first created	Screen is being built (like setting up the stage)
`onStart()`	When the Activity becomes visible	Screen is about to appear on display
`onResume()`	When the Activity comes to the foreground	User can now interact with the screen
`onPause()`	When another activity partially covers this one	Screen is losing focus (e.g., a call comes in)
`onStop()`	When the Activity is no longer visible	Screen is completely hidden (user switched app)
`onRestart()`	When the Activity is coming back after being stopped	Screen is about to be shown again
`onDestroy()`	When the Activity is finally closed	Screen is being removed completely from memory

EXAMPLE

You are filling a form (Activity running). A phone call comes in — `onPause()` is called. You end the call and come back — `onResume()` is called, and your form is still there, exactly as you left it!

Sample Activity Code

```
public class MainActivity extends AppCompatActivity {
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);   // loads the screen design
    }

    @Override
    protected void onPause() {
        super.onPause();
        // Save data here, e.g., pause a video
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        // Clean up resources here
    }
}
```

QUICK REVISION - REMEMBER THIS ORDER

Starting an Activity: onCreate → onStart → onResume
Leaving an Activity: onPause → onStop → onDestroy
Coming Back to it: onRestart → onStart → onResume

EXAM TIP

Students often forget the difference between `onPause()` and `onStop()`. Remember: **onPause** = screen partially hidden (still a little visible), **onStop** = screen fully hidden (not visible at all).

✅ Quick Summary (Revision Page)

- **Android** = Operating system for mobiles, tablets, and embedded devices.
- **SDK** = Toolbox with everything needed to build Android apps (emulator, libraries, build tools).
- **Development Features** = Ready-made building blocks (Activities, Services, Intents, etc.)
- **Mobile vs Embedded** = Phones/tablets have more resources; embedded devices (watch, TV, car) have limited resources.
- **Development Tools** = Android Studio, Emulator, ADB, Gradle, Logcat.
- **App Creation** = Plan → Create Project → Design UI → Write Code → Test → Publish.
- **App Basics** = 4 components: Activity, Service, Broadcast Receiver, Content Provider.
- **Manifest File** = App's ID card; lists activities, permissions, and app details.
- **Externalizing Resources** = Keeping text/colors/images separate from code in the res/ folder.
- **Application Lifecycle** = Foreground → Visible → Background → Killed.
- **Activity Lifecycle** = onCreate → onStart → onResume → onPause → onStop → onDestroy.